

Software Engineering 1

Abgabedokument

Teilaufgabe 1

(Anforderungsanalyse und Planungsphase)

Persönliche Daten, bitte vollständig ausfüllen:

Nachname, Vorname:	Frank, Ksenia
Matrikelnummer:	01577142
E-Mail-Adresse:	a01577142@unet.univie.ac.at
Datum:	31.03.2023

Aufgabe 1: Anforderungsanalyse

Typ der Anforderung: funktional

Anforderung 1

- **Beschreibung:** Clients Registration - *Der Client* muss sich sobald er gestartet wurde am Server registrieren, um ein Spiel anfangen zu können.
- **Bezugsquelle:** Spielidee, „Nach Start des Clients registrieren sich die KIs für das Spiel am Server.“ ; Netzwerkprotokoll, „Sobald ein Spiel erstellt wurde und beiden Clients die eindeutige SpielID bekannt ist können sich die Clients für das neue Spiel registrieren.“

Anforderung 2

- **Beschreibung:** Erstellung der Kartenhälften - *Der Client* muss nach seiner Registrierung eine Kartenhälfte erstellen, um ein Spiel anfangen zu können.
- **Bezugsquelle:** Spielidee, „Die von den KIs bespielte Karte wird bei Beginn des Spiels von beiden beteiligten KIs kooperativ erstellt. Hierzu erstellt jede der beiden KIs zufällig eine Hälfte der finalen Spielkarte.“

Anforderung 3

- **Beschreibung:** Anfrage zum Spielstatus - *Der Client* muss nach seiner Registrierung den Zustand des Spiels am Server anfragen, um Details zum Spiel erhalten zu können.
- **Bezugsquelle:** Netzwerkprotokoll, „Die Abfrage eines Spielstatus durch den Client kann erst nach der Registrierung eines Clients beginnen da hierzu die SpielID aber auch die SpielerID benötigt werden. Bei einer Abfrage über den Spielzustand werden vom Server alle für die Clients relevanten Details zum Spiel zurückgeliefert.“

Anforderung 4

- **Beschreibung:** Rückmeldung zur Abfrage des Spielstatus - *Der Server* muss bei Abfrage vom Client den Spielzustand zurückliefern, um alle Details zum Spiel mitzuteilen.
- **Bezugsquelle:** Netzwerkprotokoll, „Die Abfrage eines Spielstatus durch den Client kann erst nach der Registrierung eines Clients beginnen da hierzu die SpielID aber auch die SpielerID benötigt werden. Bei einer Abfrage über den Spielzustand werden vom Server alle für die Clients relevanten Details zum Spiel zurückgeliefert.“

Typ der Anforderung: nicht funktional

Anforderung 1

- **Beschreibung:** Schatzsuche - *Der Client* muss nach dem Kartenhälftenaustausch eigener Schatz *schnellstmöglich* finden, um ein Spiel gewinnen zu können.
- **Bezugsquelle:** Spielidee, „Nach dem Kartenhälftenaustausch starten die Spielfiguren jeder KI jeweils auf einer selbst ausgewählten Position (ihrer Burg) auf der von ihr erstellten Kartenhälfte. Anschließend muss jede KI die Karte erkunden um schnellstmöglich ihren Schatz und danach die gegnerische Burg zu finden.“

Anforderung 2

- **Beschreibung:** Aufdeckung der Kartenfelder - *Der Client* muss die Kartenfelder *schrittweise* aufdecken, um den Schatz zu finden.
- **Bezugsquelle:** Spielidee, „Um den Schatz zu finden, bewegen beide KIs ihre Spielfigur über die Karte und decken dabei mit ihrer Spielfigur schrittweise Kartenfelder auf.“

Anforderung 3

- **Beschreibung:** Kartengenerierung - *Der Server* muss die beiden Kartenhälften *zufällig* zusammenführen.
- **Bezugsquelle:** Spielidee, „Die Kartenhälften werden vom Server *zufällig* entweder an den kurzen (Kartenabmessungen 5 x 20) oder den langen Seiten (Kartenabmessungen 10 x 10) zusammengefügt.“

Anforderung 4

- **Beschreibung:** Zeitspanne zwischen Statusabfragen - Der Client muss zwischen zwei Abfragen zum Spielstatus eine Zeitspanne von *mindestens 0,4 Sekunden* warten, um Überlastung des Servers zu verhindern.
- **Bezugsquelle:** Netzwerkprotokoll, „Um zu verhindern, dass der Server überlastet wird sollte zwischen zwei vom gleichen Client durchgeführten Abfragen zum Spielstatus mindestens eine Zeitspanne von 0,4 Sekunden vergehen. “

Typ der Anforderung: Designbedingung

Anforderung 1

- **Beschreibung:** Nachrichten müssen in XML Formar definiert sein.
- **Bezugsquelle:** Netzwerkprotokoll, „Die ausgetauschten Nachrichten werden im XML Format definiert bzw. erwartet“.

Aufgabe 2: Anforderungsdokumentation - Spielstatus

- **Name:** Rückmeldung zur Abfrage des Spielstatus

- **Beschreibung und Priorität:** Der Server muss Spielstatus zurückliefern, sobald ein Client das abgefragt hat. Spielstatus umfasst Spielkarte und Informationen zur Aktion des Clients. Karteninformation kann entweder ganze Karte oder ein Teil davon sein; Aktionsinformation ist ein Erlaubnis eine Aktion auszuführen oder Mitteilung zur Gewinn oder Verlust eines Spiels.

Priorität: Hoch

- **Relevante Anforderungen:**

- ◆ Clients Registration - *Der Client* muss sich sobald er gestartet wurde am Server registrieren, um ein Spiel anfangen zu können.

- ◆ Erstellung der Kartenhälften - *Der Client* muss nach seiner Registrierung eine Kartenhälfte erstellen, um ein Spiel anfangen zu können.

- ◆ Anfrage zum Spielstatus - *Der Client* muss nach seiner Registrierung den Zustand des Spiels am Server anfragen, um Details zum Spiel erhalten zu können.

- ◆ Rückmeldung zur Abfrage des Spielstatus - *Der Server* muss bei Abfrage vom Client den Spielzustand zurückliefern, um alle Details zum Spiel mitzuteilen.

- **Relevante Business Rules:**

- ◆ Ein Spiel darf nicht länger als 320 Spielaktionen (und damit 320 Runden) dauern.

- ◆ Für jede der rundenbasierten Spielaktion hat die KI maximal 5 Sekunden Bedenkzeit.

- ◆ Es muss zwischen zwei vom gleichen Client durchgeführten Abfragen zum Spielstatus mindestens eine Zeitspanne von 0,4 Sekunden vergehen.

• **Impuls/Ergebnis - Typisches Szenario:**

• **Vorbedingungen:**

- ◆ Ein Spiel ist erzeugt.
- ◆ Beide Clients sind für das Spiel registriert.
- ◆ Sowohl *SpielID* als auch *SpielerID* sind den Clients bekannt.
- ◆ Der erste Client hat Kartenhälfte schon erstellt.
- ◆ Der zweite Client hat Kartenhälfte noch nicht erstellt.

Hauptsächlicher Ablauf:

- ◆ Impuls: Der erste Client überträgt die Kartenhälfte in einer Nachricht an Server.
- ◆ Ergebnis: Der Server überprüft die Kartenhälfte und sendet den Client eine *responseEnvelope* mit *state Okay*.
- ◆ Impuls: Der erste Client sendet eine Nachricht mit Abfrage zum Spielstatus an Server erstmal.
- ◆ Ergebnis: Der Server stellt die Nutzdaten der Clients zusammen und antwortet mit einem *responseEnvelope* mit Kartenhälfte und *state „MustWait“*.
- ◆ Impuls: Der zweite Client überträgt seine Kartenhälfte in einer Nachricht an Server.
- ◆ Ergebnis: Der Server überprüft die Kartenhälfte und antwortet den zweiten Client mit einem *responseEnvelope* mit *state Okay*.
- ◆ Impuls: Der erste Client sendet zweite Nachricht mit Abfrage zum Spielstatus an Server nach 0.4 Sekunden.
- ◆ Ergebnis: Server überprüft, ob der Client eine Aktion ausführen darf. Server fasst die Nutzdaten der Client zusammen und antwortet mit einem *responseEnvelope* mit *State „MustAct“*.
- ◆ Impuls: Der zweite Client sendet eine Nachricht mit Abfrage zum Spielstatus an Server.
- ◆ Ergebnis: Der Server stellt die Nutzdaten der zweiten Clients zusammen und antwortet mit einem *responseEnvelope* mit teilweise Karte und *state „MustWait“*.

Nachbedingungen:

- ◆ Der Server überprüft, ob die Spielbewegung Fehlerfrei ist und antwortet den Client mit Status **Okay**.
- ◆ Der Server aktualisiert die gespeicherte Karte.

• **Impuls/Ergebnis - Alternativszenario:**

Vorbedingungen:

- ◆ Ein Spiel ist erzeugt.
- ◆ Beide Clients sind für das Spiel registriert.
- ◆ Sowohl *SpielID* als auch *SpielerID* sind den Clients bekannt.
- ◆ Der Server hat die Karte aus beiden Kartenhälften schon erstellt.
- ◆ Erste 20 Spielaktionen sind schon ausgeführt.
- ◆ Der erste Client ist in der Reihe.

Hauptsächlicher Ablauf:

- ◆ Impuls: Der erste Client überträgt eine Nachricht mit Spielbewegung an Server.
- ◆ Ergebnis: Der Server überprüft die Bewegung und sendet dem ersten Client eine *responseEnvelope* mit state **Okay**. Danach aktualisiert der Server die gespeicherte Spielkarte.
- ◆ Impuls: Der zweite Client sendet eine Nachricht mit Abfrage zum Spielstatus an Server.
- ◆ Ergebnis: Der Server stellt die Nutzdaten der Clients zusammen und antwortet mit einem *responseEnvelope* mit aktualisierter Karte und state „*MustAct*“.
- ◆ Impuls: Der erste Client sendet eine Nachricht mit Abfrage zum Spielstatus an Server.
- ◆ Ergebnis: Der Server stellt die Nutzdaten der zweiten Clients zusammen und antwortet mit einem *responseEnvelope* mit state „*MustWait*“.
- ◆ Impuls: Der zweite Client überträgt eine Nachricht mit Spielbewegung in die Richtung des Schatzes an Server.
- ◆ Ergebnis: Der Server überprüft die Bewegung und aktualisiert die Karte indem er auch den Schatz des zweiten Client von der Karte entfernt.
- ◆ Impuls: Der erste Client sendet nächste Nachricht mit Abfrage zum Spielstatus an Server.
- ◆ Ergebnis: Server überprüft, ob der Client eine Aktion ausführen darf. Server fasst die Nutzdaten der Client zusammen und antwortet mit einem *responseEnvelope* mit State „*MustAct*“.

Nachbedingungen:

- ◆ Der erste Client überträgt nächste Spielbewegung an den Server.
- ◆ Der Server überprüft, ob die Spielbewegung Fehlerfrei ist und antwortet den Client mit Status **Okay**.
- ◆ Der Server aktualisiert die gespeicherte Karte.

• **Impuls/Ergebnis - Fehlerfall:**

Vorbedingungen:

- ◆ Ein Spiel ist erzeugt.
- ◆ Beide Clients sind für das Spiel registriert.
- ◆ Sowohl *SpielID* als auch *SpielerID* sind den Clients bekannt.
- ◆ Erster Client hat Kartenhälfte schon erstellt.
- ◆ Zweiter Client hat Kartenhälfte noch nicht erstellt.

Hauptsächlicher Ablauf:

- ◆ Impuls: Der erste Client sendet eine Nachricht mit Abfrage zum Spielstatus an Server.
- ◆ Ergebnis: Der Server stellt die Nutzdaten der zweiten Clients zusammen und antwortet mit einem *responseEnvelope* mit Kartenhälfte und state „*MustWait*“.
- ◆ Impuls: Der erste Client überträgt die Kartenhälfte in einer Nachricht an Server.
- ◆ Ergebnis: Der Server überprüft die Kartenhälfte und findet heraus, dass die Spielregel verletzt ist, indem die von Client übertragene Kartenhälfte eine Insel beinhaltet. Der Server antwortet den zweiten Client mit einem *responseEnvelope* mit state **Error**, Details zum Fehler in *exceptionName* und *exceptionsMessage* Felder und *PlayerGameState* „Lost“. Der Server aktualisiert der gespeicherte Zustand des Spiels.
- ◆ Der zweite Client sendet eine Nachricht mit Abfrage zum Spielstatus an Server.
- ◆ Der Server stellt die Nutzdaten der zweiten Clients zusammen und antwortet mit einem *responseEnvelope* mit Teil der Karte und state „*Won*“.

Nachbedingungen:

- ◆ CLI/GUI visualisieren den letzten Spielzustand für Clients. Bei ersten Client wird Spielzustand „Verloren“ ausgegeben und bei zweiten Client Spielzustand „Gewonnen“.

- ◆ Der Server entfernt das Spiel und alle dazugehörigen Daten.
- ◆ Menschliche Spieler haben die Möglichkeit ein neues Spiel zu erstellen.

• **Benutzergeschichten:**

- ◆ Als Anwender möchte ich, dass alle Details zur Spielstatus richtig und rechtzeitig übertragen und aktualisiert sind, damit alle relevanten Spieldaten mithilfe CLI/GUI ausgegeben werden können.
- ◆ Als Client möchte ich Spielstatus anfragen können, damit ich alle wichtigen Daten habe.
- ◆ Als KI möchte ich Spielstatus anfragen können, damit ich die aktuellen Daten in meinen Algorithmen benutzen kann.
- ◆ Als Server möchte ich, dass die Anfragen zur Spielstatus von einem Client nicht öfter als einmal in 0.4 Sekunden wiederholt sind, damit ich nicht überlastet werde.
- ◆ Als Client möchte ich, dass der Server die richtigen Daten auf meine Statusabfrage schickt, damit ich richtige Aktionen anführen kann und ein Spiel nicht verliere.

• **Benutzerschnittstelle:**

Ausgabe der Daten des Spielstatus auf die CLI Benutzeroberfläche erfolgen in String format auf Englisch wie folgt:

- ◆ Ausgabe der Daten des menschlichen Spielers (Vorname, Nachname, uaccount):

Vollständige name und uaccount.

Beispiel:

Name: Max Mustermann uaccount: 0000000

- ◆ Ausgabe des Spielfortschritts:

Treasure found: YES
Action: MAKES MOVE

♦ Ausgabe der Karte:

Um Spielkarte zu repräsentieren müssen folgende emojis verwendet werden:

♦ Für Terrainarten verschiedene Farben:

-  für ein Grasfeld
-  für ein Wasserfeld
-  für ein Bergfeld

♦ Für Spielfiguren:

- 
- 

♦ Ein Burg:

♦ Ein Schatz:

• **Externe Schnittstellen:**

1. Für Nachrichtenaustausch zwischen Client und Server wird Restschnittstelle und daher HTTP Protokoll mit zugehörigen Operationen GET und POST.

♦ Um ein Spiel zu erstellen muss eine HTTP GET Operation an `http(s)://<domain>:<port>/games` Endpoint gesendet werden.

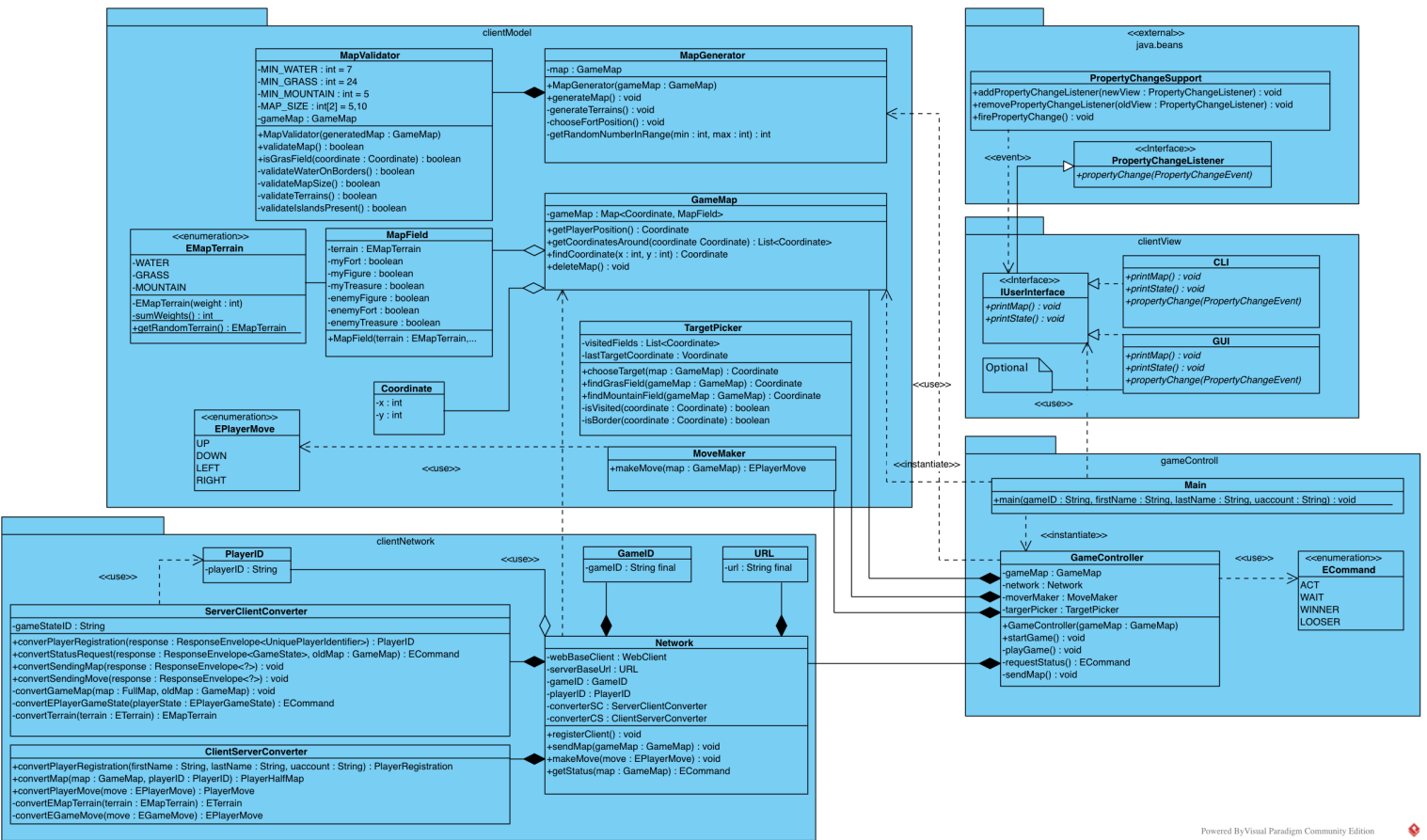
♦ Um sich für ein Spiel zu registrieren, muss ein Client einen HTTP POST Request an `http(s)://<domain>:<port>/games/<SpielID>/players` senden.

♦ Um eigene Kartenhälfte an Server zu übertragen, muss ein Client einen HTTP POST Request an `http(s)://<domain>:<port>/games/<SpielID>/halfmaps` Endpoint senden.

♦ Um den Spielstatus anzufragen muss ein Client einen HTTP GET Request an `http(s)://<domain>:<port>/games/<SpielID>/states/<SpielerID>` senden.

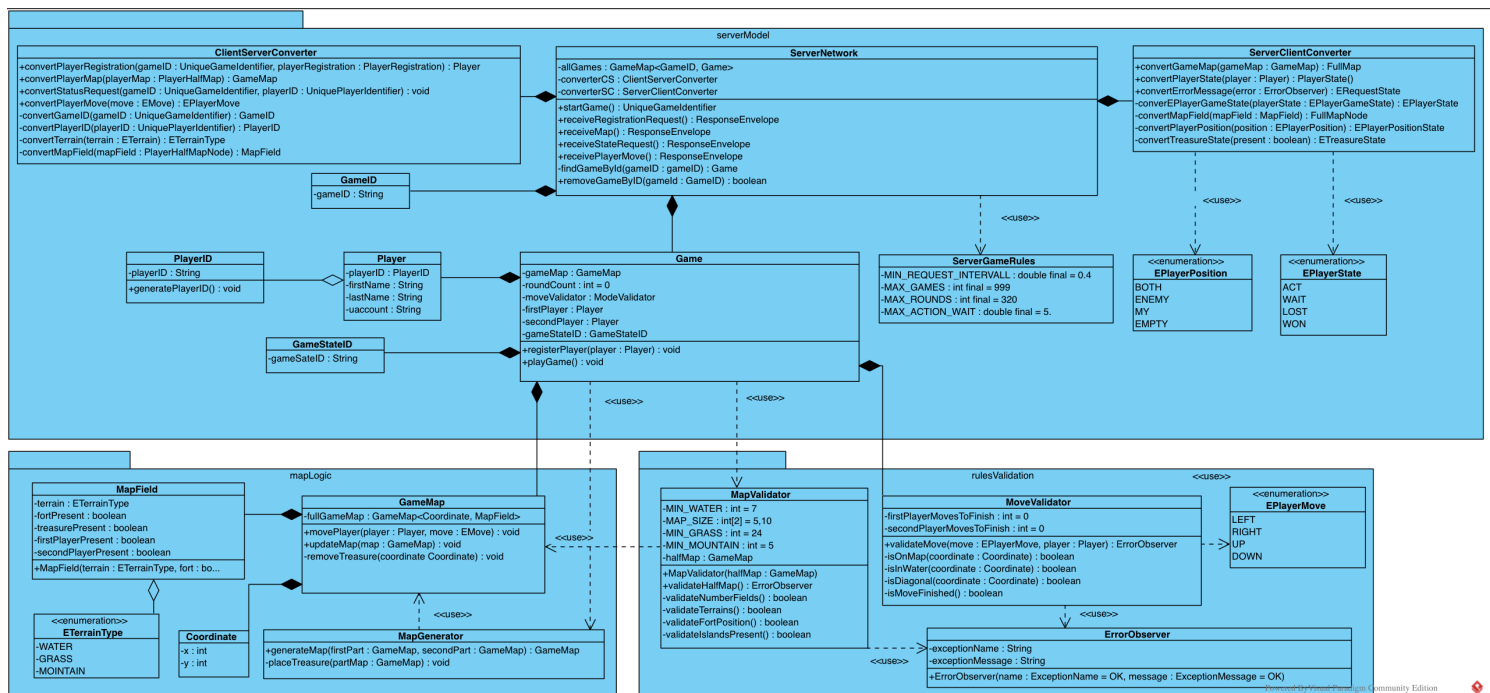
♦ Um eine Bewegung an Server zu übertragen muss ein Client einen HTTP POST Request an `http(s)://<domain>:<port>/games/<SpielID>/moves` Endpoint senden.

Client Klassendiagramm



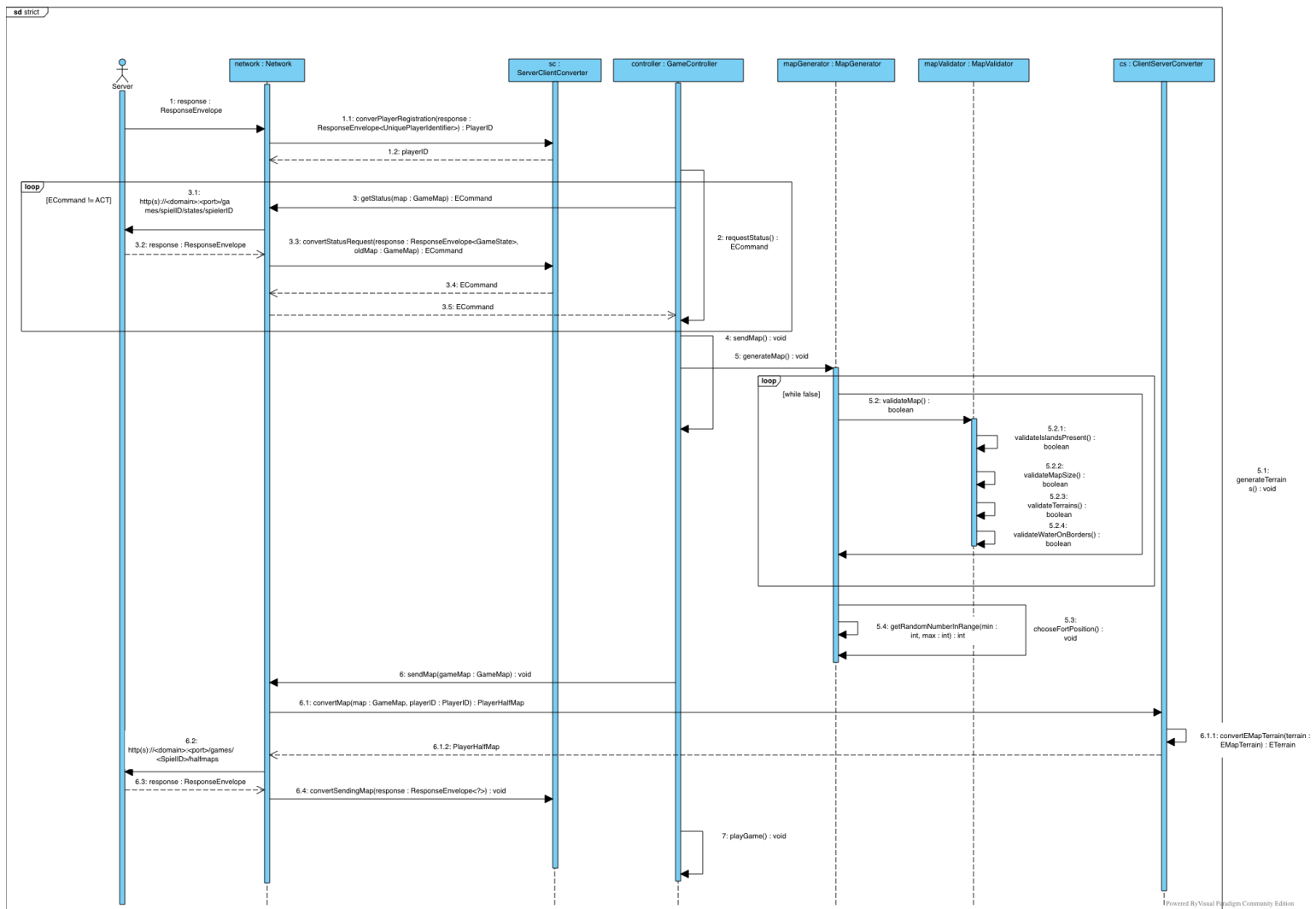
Powered By Visual Paradigm Community Edition

Server Klassendiagramm



Powered By Visual Paradigm Community Edition

Client Sequenzdiagramm



Server Sequenzdiagramm

